

第四章 语法分析

Syntax Analysis

Part II

4、LL(1)预测分析法

- 常用终结符号集计算
- LL(1)文法
- LL(1)预测分析表
- LL(1)预测分析方法

1) 常用终结符号集

- $\text{First}(\alpha)$
- $\text{Follow}(A)$
- $\text{Select}(A \rightarrow \beta)$

FIRST集 和 FOLLOW集

- 对于 $\forall \alpha \in (V_T \cup V_N)^*$, 定义 α 的首符号集
 - $\text{FIRST}(\alpha) = \{a \mid \alpha \Rightarrow^* a \dots, a \in V_T\}$
 - 若 $\alpha \Rightarrow^* \varepsilon$, 则规定 $\varepsilon \in \text{FIRST}(\alpha)$
- 对于 $\forall A \in V_N$, 定义 A 的后续符号集
 - $\text{FOLLOW}(A) = \{a \mid S \Rightarrow^* \dots Aa \dots, a \in V_T\}$

SELECT集

- 如果 $\beta \xRightarrow{*} \varepsilon$
 - **$\text{Select}(A \rightarrow \beta) = \text{First}(\beta) \setminus \{ \varepsilon \} \cup \text{Follow}(A)$**
- 否则
 - **$\text{Select}(A \rightarrow \beta) = \text{First}(\beta)$**

例子

- $S \rightarrow pA$
- $S \rightarrow qB$
- $A \rightarrow cAd$
- $A \rightarrow a$

$w = pccadd$

例子

- $S \rightarrow Ap$
- $S \rightarrow Bq$
- $A \rightarrow a$
- $A \rightarrow cA$
- $B \rightarrow b$
- $B \rightarrow dB$

$w_1 = ccap$

$w_2 = ddbq$

例子

- $S \rightarrow aA$
- $S \rightarrow d$
- $A \rightarrow bAS$
- $A \rightarrow \epsilon$

$w = abd$

求FIRST(X) 的算法

1) 对 $\forall X \in V_T$, $\text{FIRST}(X) = \{X\}$

2) 对 $\forall X \in V_N$, 设置 $\text{FIRST}(X)$ 的初值:

$$\text{FIRST}(X) = \begin{cases} \{a | X \rightarrow a... \in P\} & X \rightarrow \varepsilon \notin P \\ \{a | X \rightarrow a... \in P\} \cup \{\varepsilon\} & X \rightarrow \varepsilon \in P \end{cases}$$

3) 对 $\forall X \in V_N$, 重复如下过程, 直到所有FIRST集不变

若 $X \rightarrow Y... \in P$, 且 $Y \in V_N$,

则 $\text{FIRST}(X) = \text{FIRST}(X) \cup (\text{FIRST}(Y) - \{\epsilon\})$

若 $X \rightarrow Y_1...Y_k \in P$, 且 $Y_1...Y_{i-1} \Rightarrow^* \epsilon$,

则 对 $n=1$ 到 $i-1$:

$\text{FIRST}(X) = \text{FIRST}(X) \cup (\text{FIRST}(Y_n) - \{\epsilon\})$

若 $Y_1...Y_k \Rightarrow^* \epsilon$,

则 $\text{FIRST}(X) = \text{FIRST}(X) \cup \{\epsilon\}$

例：表达式文法语法符号的FIRST集

FIRST(F)={ (, id }

FIRST(T)=FIRST(F)={ (, id }

FIRST(E)=FIRST(T)={ (, id }

FIRST(E')={ +, ϵ }

FIRST(T')={ *, ϵ }

FIRST(+)=\{+\}, FIRST(*)=\{*\}

FIRST(()=\{ (}

FIRST())=\{) }

FIRST(id)=\{id\}

$E \rightarrow TE'$

$E' \rightarrow +TE' | \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' | \epsilon$

$F \rightarrow (E) | id$

求FOLLOW(B)的算法

1) 先将 # 加入到 FOLLOW(S)

– #为句子结束符

对于所有非终结符，重复进行以下计算：

2) 若 $A \rightarrow \alpha B \beta$,

– 则 $\text{FOLLOW}(B) = \text{FOLLOW}(B) \cup \text{FIRST}(\beta) - \{\epsilon\}$

3) 若 $A \rightarrow \alpha B$ 或 $A \rightarrow \alpha B \beta$, 且 $\beta \Rightarrow^* \epsilon$, $A \neq B$,

– 则 $\text{FOLLOW}(B) = \text{FOLLOW}(B) \cup \text{FOLLOW}(A)$

例：表达式文法语法符号的 FOLLOW 集

FIRST(F) = { (, id }

$E \rightarrow TE'$

FIRST(T) = { (, id }

$E' \rightarrow +TE' | \varepsilon$

FIRST(E) = { (, id }

$T \rightarrow FT'$

FIRST(E') = { +, ε }

$T' \rightarrow *FT' | \varepsilon$

FIRST(T') = { *, ε }

$F \rightarrow (E) | id$

FOLLOW(E) = {), # }

FOLLOW(E') = FOLLOW(E) = {), # }

FOLLOW(T) = { +,), # }

FOLLOW(T') = FOLLOW(T) = { +,), # }

FOLLOW(F) = { *, +,), # }

例：表达式文法产生式的 SELECT集

FIRST(F)= { (, id }

FOLLOW(E) = {), # }

FIRST(T)= { (, id }

FOLLOW(E') = {), # }

FIRST(E)= { (, id }

FOLLOW(T) = { +,), # }

FIRST(E')= { +, ϵ }

FOLLOW(T') = { +,), # }

FIRST(T')= { *, ϵ }

FOLLOW(F) = { *, +,), # }

SELECT(E $\rightarrow$ TE')= { (, id }

E $\rightarrow$ TE'

SELECT(E' $\rightarrow$ +TE')= { + }

E' $\rightarrow$ +TE' | ϵ

SELECT(E' $\rightarrow$ ϵ)= {), # }

T $\rightarrow$ FT'

SELECT(T $\rightarrow$ FT')= { (, id }

T' $\rightarrow$ *FT' | ϵ

SELECT(T' $\rightarrow$ ϵ)= { +,), # }

F $\rightarrow$ (E) | id

Review

- $\text{First}(A)$
- $\text{Follow}(A)$

The *first set* of a sequence of symbols u , written as $\text{First}(u)$ is the set of terminals which start all the sequences of symbols derivable from u . A bit more formally, consider all strings derivable from u by a leftmost derivation. If $u \Rightarrow^* v$, where v begins with some terminal, that terminal is in $\text{First}(u)$. If $u \Rightarrow^* \varepsilon$, then ε is in $\text{First}(u)$.

The *follow set* of a nonterminal A is the set of terminal symbols that can appear immediately to the right of A in a valid sentence. A bit more formally, for every valid sentence $S \Rightarrow^* uAv$, where v begins with some terminal, that terminal is in $\text{Follow}(A)$.

To calculate $\text{First}(u)$ where u has the form $X_1X_2\dots X_n$, do the following:

1. If X_1 is a terminal, then add X_1 to $\text{First}(u)$, otherwise add $\text{First}(X_1) - \varepsilon$ to $\text{First}(u)$.
2. If X_1 is a nullable nonterminal, i.e., $X_1 \Rightarrow^* \varepsilon$, add $\text{First}(X_2) - \varepsilon$ to $\text{First}(u)$. Furthermore, if X_2 can also go to ε , then add $\text{First}(X_3) - \varepsilon$ and so on, through all X_n until the first nonnullable one.
3. If $X_1X_2\dots X_n \Rightarrow^* \varepsilon$, add ε to the First set.

Follow()

1. Place # in Follow(S) where S is the start symbol and # is the input's right endmarker. The endmarker might be end of file, it might be newline, it might be a special symbol, whatever is the expected end of input indication for this grammar. We will typically use # as the endmarker.
2. For every production $A \rightarrow uBv$ where u and v are any string of grammar symbols and B is a nonterminal, everything in First(v) except ϵ is placed in Follow(B).
3. For every production $A \rightarrow uB$, or a production $A \rightarrow uBv$ where First(v) contains ϵ (i.e. v is nullable), then everything in Follow(A) is added to Follow(B).

例:

- $S \rightarrow aAd$
- $A \rightarrow BC$
- $B \rightarrow b \mid \epsilon$
- $C \rightarrow c \mid \epsilon$

求First(A), Follow(B)

例:

- $S \rightarrow A B$
- $A \rightarrow B a \mid \varepsilon$
- $B \rightarrow D b \mid D$
- $D \rightarrow d \mid \varepsilon$

$\text{Select}(S \rightarrow AB)=?$

2) LL(1)文法

Top-Down Parsing

- 从左到右扫描输入串——寻找它的一个最左推导
- 文法G是 LL(1)文法的充要条件
 - 对于G 的每个非终结符 A 的任何两个不同的候选式 $A \rightarrow \alpha | \beta$, 满足:
 - (1) $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \phi$
 - (2) 如果 $\beta \Rightarrow^* \varepsilon$, 则 $\text{FISRT}(\alpha) \cap \text{FOLLOW}(A) = \phi$

- LL(1)文法表示自顶向下方法能够处理的一类文法
 - 第一个 **L** 表示从左向右扫描输入符号串
 - 第二个 **L** 表示生成最左推导
 - **1** 表示读入一个符号可确定下一步推导

表达式文法是 LL(1) 文法

- $E \rightarrow T E'$
- $E' \rightarrow + T E' \mid \varepsilon$
- $T \rightarrow F T'$
- $T' \rightarrow * F T' \mid \varepsilon$
- $F \rightarrow (E) \mid \text{id}$

判断如下:

- E' : $+$ 不在 $\text{FOLLOW}(E') = \{), \# \}$
- T' : $*$ 不在 $\text{FOLLOW}(T') = \{ +,), \# \}$
- F : $($ 和 id 不同

文法是否为LL(1) ?

G[S]:

- $S \rightarrow AB \mid bC$
- $A \rightarrow b \mid \epsilon$
- $B \rightarrow aD \mid \epsilon$
- $C \rightarrow AD \mid b$
- $D \rightarrow aS \mid c$

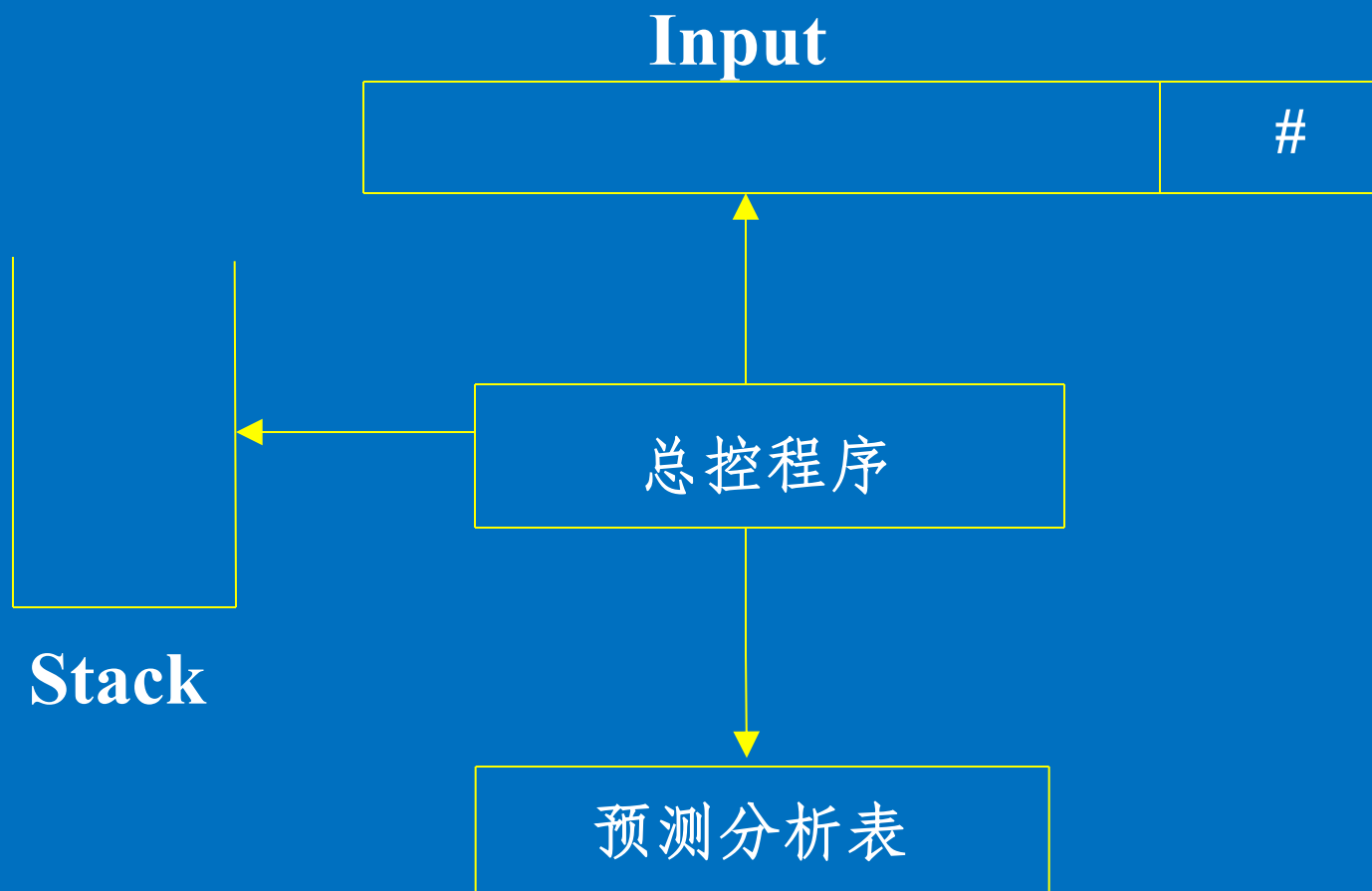
针对LL(1)文法的统一的控制算法

- 通过一个统一的算法实现语法分析
- 根据以下两个因素决定候选产生式
 - 栈顶符号（当前的语法变量）
 - 输入符号

预测分析方法

- 系统维持一个分析表和一个分析栈，根据当前扫描到的符号，选择当前语法变量（处于栈顶）的候选式进行推导——找到相应的输入符号串的最左推导
- 一个通用的控制算法
- 一个统一形式的分析表M
 - 不同语言使用内容不同的分析表
- 一个分析栈
 - #为栈底符号
- 一个输入缓冲区
 - #为输入串结束符

表驱动预测分析程序模型



表达式文法的预测分析表

非终结符	输入符号					
	id	+	*	(	)	#
E	$\rightarrow TE'$			$\rightarrow TE'$		
E'		$\rightarrow +TE'$			$\rightarrow \epsilon$	$\rightarrow \epsilon$
T	$\rightarrow FT'$			$\rightarrow FT'$		
T'		$\rightarrow \epsilon$	$\rightarrow *FT'$		$\rightarrow \epsilon$	$\rightarrow \epsilon$
F	$\rightarrow id$			$\rightarrow (E)$		

3) LL(1)分析表的构造

LL(1)分析表的构造算法I

- 1) 对于每一产生式 $A \rightarrow \alpha$, 执行2)
- 2) 对于 $\text{Select}(A \rightarrow \alpha)$ 中的每一终结符 a , 将 $A \rightarrow \alpha$ 填入 $M[A, a]$
- 3) 将所有无定义的 $M[A, b]$ 标上错误标志

- G[E]:**
- (1) $E \rightarrow TE'$ (2) $E' \rightarrow +TE'$ (3) $E' \rightarrow \varepsilon$
 - (4) $T \rightarrow FT'$ (5) $T' \rightarrow *FT'$ (6) $T' \rightarrow \varepsilon$
 - (7) $F \rightarrow (E)$ (8) $F \rightarrow id$

	id	+	*	(	)	#
E	$\rightarrow TE'$			$\rightarrow TE'$		
E'						
T						
T'						
F						

Select($E \rightarrow TE'$) = {(, id}

- G[E]:**
- (1) $E \rightarrow TE'$ (2) $E' \rightarrow +TE'$ (3) $E' \rightarrow \epsilon$
 - (4) $T \rightarrow FT'$ (5) $T' \rightarrow *FT'$ (6) $T' \rightarrow \epsilon$
 - (7) $F \rightarrow (E)$ (8) $F \rightarrow id$

	id	+	*	(	)	#
E	$\rightarrow TE'$			$\rightarrow TE'$		
E'		$\rightarrow +TE'$			$\rightarrow \epsilon$	$\rightarrow \epsilon$
T						
T'						
F	Select($E' \rightarrow +TE'$) = {+}					

Select($E' \rightarrow \epsilon$) = { }, #}

G[E]: (1) $E \rightarrow TE'$ (2) $E' \rightarrow +TE'$ (3) $E' \rightarrow \varepsilon$
 (4) $T \rightarrow FT'$ (5) $T' \rightarrow *FT'$ (6) $T' \rightarrow \varepsilon$
 (7) $F \rightarrow (E)$ (8) $F \rightarrow id$

	id	+	*	(	)	#
E	$\rightarrow TE'$			$\rightarrow TE'$		
E'		$\rightarrow +TE'$			$\rightarrow \varepsilon$	$\rightarrow \varepsilon$
T	$\rightarrow FT'$			$\rightarrow FT'$		
T'						
F						

Select($T \rightarrow FT'$) = { (, id }

G[E]: (1) $E \rightarrow TE'$ (2) $E' \rightarrow +TE'$ (3) $E' \rightarrow \varepsilon$
 (4) $T \rightarrow FT'$ (5) $T' \rightarrow *FT'$ (6) $T' \rightarrow \varepsilon$
 (7) $F \rightarrow (E)$ (8) $F \rightarrow id$

	id	+	*	(	)	#
E	$\rightarrow TE'$			$\rightarrow TE'$		
E'		$\rightarrow +TE'$			$\rightarrow \varepsilon$	$\rightarrow \varepsilon$
T	$\rightarrow FT'$			$\rightarrow FT'$		
T'		$\rightarrow \varepsilon$	$\rightarrow *FT'$		$\rightarrow \varepsilon$	$\rightarrow \varepsilon$
F	$\rightarrow id$			$\rightarrow (E)$		

Select($T' \rightarrow \varepsilon$) = {+,), #}

G[E]: (1) $E \rightarrow TE'$ (2) $E' \rightarrow +TE'$ (3) $E' \rightarrow \varepsilon$
 (4) $T \rightarrow FT'$ (5) $T' \rightarrow *FT'$ (6) $T' \rightarrow \varepsilon$
 (7) $F \rightarrow (E)$ (8) $F \rightarrow id$

	id	+	*	(	)	#
E	$\rightarrow TE'$			$\rightarrow TE'$		
E'		$\rightarrow +TE'$			$\rightarrow \varepsilon$	$\rightarrow \varepsilon$
T	$\rightarrow FT'$			$\rightarrow FT'$		
T'		$\rightarrow \varepsilon$	$\rightarrow *FT'$		$\rightarrow \varepsilon$	$\rightarrow \varepsilon$
F	$\rightarrow id$			$\rightarrow (E)$		

表达式文法的预测分析表

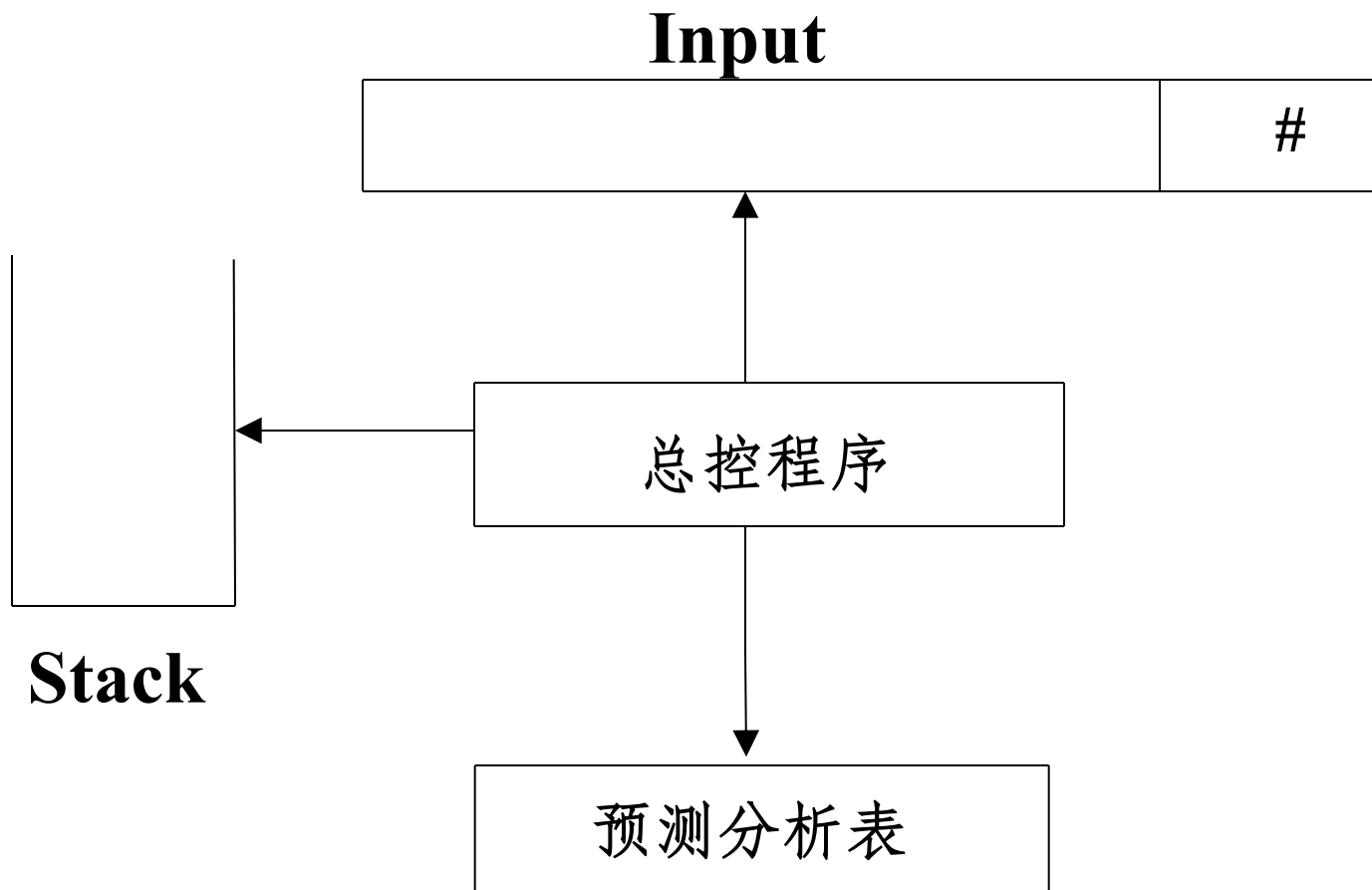
非终结符	输入符号					
	id	+	*	(	)	#
E	$\rightarrow TE'$			$\rightarrow TE'$		
E'		$\rightarrow +TE'$			$\rightarrow \epsilon$	$\rightarrow \epsilon$
T	$\rightarrow FT'$			$\rightarrow FT'$		
T'		$\rightarrow \epsilon$	$\rightarrow *FT'$		$\rightarrow \epsilon$	$\rightarrow \epsilon$
F	$\rightarrow id$			$\rightarrow (E)$		

预测分析表构造算法II

- 1.对文法G的每个产生式 $A \rightarrow \alpha$ 执行第二步和第三步;
- 2.对每个终结符 $a \in \text{FIRST}(\alpha)$, 把 $A \rightarrow \alpha$ 加至 $M[A, a]$ 中;
- 3.若 $\varepsilon \in \text{FIRST}(\alpha)$, 则对任何 $b \in \text{FOLLOW}(A)$ 把 $A \rightarrow \alpha$ 加至 $M[A, b]$ 中;
- 4.把所有无定义的 $M[A, a]$ 标上“出错标志”。

4) LL(1)分析法

表驱动预测分析程序模型



系统执行及特点

- 系统启动时，输入指针指向输入串的第一个字符，分析栈中存放着栈底符号#和文法的开始符号
- 根据栈顶符号 A 和读入的符号 a ，查看分析表 M ，以决定相应的动作
- 优点：
 - 1) 效率高
 - 2) 便于维护、自动生成
- 关键：
 - 分析表 M 的构造

LL(1) 通用控制算法

repeat

X =当前栈顶符号; a =当前输入符号;

if $X \in V_T \cup \{\#\}$ *then*

if $X=a$ *then*

if $X \neq \#$ *then* {将 X 弹出,且前移输入指针}

else error

else

if $M[X,a]=Y_1Y_2\dots Y_k$ *then*

{将 X 弹出;依次将 $Y_k\dots Y_2Y_1$ 压入栈;

输出产生式 $X \rightarrow Y_1Y_2\dots Y_k$ }

else error

until $X=\#$

执行例：分析 $\text{id}+\text{id}*\text{id}$

栈	输入缓冲区	输出
#E	id+id*id#	
#E'T	id+id*id#	$E \rightarrow TE'$
#E'T'F	id+id*id#	$T \rightarrow FT'$
#E'T'id	id+id*id#	$F \rightarrow \text{id}$
#E'T'	+id*id#	
#E'	+id*id#	$T' \rightarrow \varepsilon$
#E'T+	+id*id#	$E' \rightarrow +TE'$
#E'T	id*id#	

#E'T	id*id#	
#E'T'F	id*id#	$T \rightarrow FT'$
#E'T'id	id*id#	$F \rightarrow id$
#E'T'	*id#	
#E'T'F*	*id#	$T' \rightarrow *FT'$
#E'T'F	id#	
#E'T'id	id#	$F \rightarrow id$
#E'T'	#	
#E'	#	$T' \rightarrow \varepsilon$
#	#	$E' \rightarrow \varepsilon$

输出的产生式序列形成了最左推导对应的分析树

- $\#i*(i+i)\#$

栈

$\#E$

输入

$i*(i+i)\#$

LL(1)分析中的一种错误处理办法

发现错误:

1. 栈顶的终结符与当前输入符**不匹配**
2. 非终结符A于栈顶，面临的输入符为a，但分析表M的 **$M[A,a]$ 为空**

“应急”恢复策略:

策略: 跳过输入串中的一些符号直至遇到“同步符号”为止
同步符号的选择:

1. 把**FOLLOW(A)**中的所有符号作为A的同步符号，跳过输入串中的一些符号直至遇到这些“同步符号”，把A从栈中弹出，可使分析继续
2. 把**FIRST(A)**中的符号加到A的同步符号集，当**FIRST(A)**中的符号在输入中出现时，可根据A恢复分析

LL(1)预测分析法(小结)

- 消除左递归、提取左因子
- 求每个候选式的FIRST集和非终结符的FOLLOW集
- 检查是否为 LL(1) 文法
 - 若不是 LL(1),说明文法复杂性超过LL(1) 的分析能力
- 构造预测分析表
- 实现预测分析器

LL(1)预测分析法(小结)

- 存在问题
 - 对于某些语言，难以用 LL(1) 文法来描述
 - 消除左递归和提取左因子影响文法的可读性，造成语义处理的困难

Review

- LL(1) grammar
- Recursive-descent
- Table-driven LL(1) parsing

Predictive parser and LL(1) grammar

Predictive parser is a non-backtracking top-down parser. A predictive parser is characterized by its ability to choose the production to apply solely on the basis of the next input symbol and the current nonterminal being processed.

To enable this, the grammar must take a particular form. We call such a grammar *LL(1)*. The first “L” means we scan the input from left to right; the second “L” means we create a leftmost derivation; and the 1 means one input symbol of lookahead.

Recursive-descent

The first technique for implementing a predictive parser is called *recursive-descent*.

A recursive descent parser consists of several small functions(procedures), one for each nonterminal in the grammar. As we parse a sentence, we call the functions (procedures) that correspond to the left side nonterminal of the productions we are applying. If these productions are recursive, we end up calling the functions recursively.

Table-driven LL(1) parsing

In a recursive-descent parser, the production information is embedded in the individual parse functions for each nonterminal and the run-time execution stack is keeping track of our progress through the parse. There is another method for implementing a predictive parser that uses a table to store that production along with an explicit stack to keep track of where we are in the parse.

How a table-driven predictive parser works

We push the start symbol on the stack and read the first input token. As the parser works through the input, there are the following possibilities for the top stack symbol X and the input token nonterminal a :

1. If $X = a$ and $a = \text{end of input } (\#)$: parser halts and parse completed successfully
2. If $X = a$ and $a \neq \#$: successful match, pop X and advance to next input token. This is called a *match* action.
3. If $X \neq a$ and X is a nonterminal, pop X and consult table at $[X, a]$ to see which production applies, push right side of production on stack. This is called a *predict* action.
4. If none of the preceding cases applies or the table entry from step 3 is blank, there has been a parse error

Constructing the parse table

1. For each production $A \rightarrow u$ of the grammar, do steps 2 and 3
2. For each terminal a in $\text{First}(u)$, add $A \rightarrow u$ to $M[A, a]$
3. If ε in $\text{First}(u)$, (i.e. A is nullable) add $A \rightarrow u$ to $M[A, b]$ for each terminal b in $\text{Follow}(A)$,
If ε in $\text{First}(u)$, and $\#$ is in $\text{Follow}(A)$, add $A \rightarrow u$ to $M[A, \#]$
4. All undefined entries are errors

LL(1) grammar

A grammar G is LL(1) iff whenever $A \rightarrow u \mid v$ are two distinct productions of G , the following conditions hold:

- for no terminal a do both u and v derive strings beginning with a (i.e. first sets are disjoint)
- at most one of u and v can derive the empty string
- if $v \Rightarrow^* \varepsilon$ then v does not derive any string beginning with a terminal in $\text{Follow}(A)$

Error-reporting and recovery

An error is detected in predictive parsing
when the terminal on top of the stack does
not match the next input symbol or
when nonterminal A is on top of the stack, a
is the next input symbol and the parsing
table entry $M[A, a]$ is empty.

习题

- 1、P.99 第1题 3),4)
- 2、P.100 第3题
- 3、P.101 第7题 1),3)